COMP 018: Database Administration — Activity 5 Progress Report

Database Maintenance, Performance Monitoring, and Query Analysis

Project Identification & Metadata

Course	COMP 018 Database Administration
Evaluating Group	Group 4: Olivarez, Charles Kean E., Infantado, Gerwin G., Pocsidio, Jhon Louie F., Abangan, SJ Kaizen M., Bartolata, Redge Raindel N.
Evaluating Group Repository	https://github.com/Nycri/brewpos
Target Project Monitored	BoardTrack: Boarding House & Rental Management System
Target Project Group	Group 11: Barcarse, Adrian; Cataluña, Zyruz Gabriel; Camu, Edgar; Policarpio, Kurt; Crucillo, Karl Robert
Target Project Repository	https://github.com/karlrobertc-art/BoardTrack
Primary Database	boardtrack_db (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	boardtrack_db_restore_test

ACTIVITY 5: Database Maintenance and Monitoring

2.1 Database Health & Overview Inspection

Database health inspection was conducted on PostgreSQL 18.6 running on a 64-bit Windows environment. Analysis confirmed that the database occupies a total disk footprint of approximately 9.1 megabytes across eight base tables, one view, and eight sequences within the public schema. Table storage analysis showed that room and payment account for 48 kilobytes each, lease and maintenance_request account for 40 kilobytes each, and property, tenant_profile, app_user, and user_role account for 24 to 32 kilobytes each.

2.2 Connection, Session & Activity Monitoring

Database statistics recorded one active connection with 62 committed transactions and 5 rolled back transactions. Zero lock conflicts and zero deadlocks

were recorded across sessions, confirming stable transaction processing. The rolled back transaction count reflected expected error conditions intentionally triggered during negative security verification testing in Activity 4. System catalog queries confirmed that active roles and memberships accurately reflect access control rules.

2.3 Table Statistics & Bloat Inspection

Pre-maintenance table statistics revealed dead tuple accumulation within the room, lease, and payment tables resulting from room occupancy synchronization triggers, lease status updates, and monthly rental payment postings executed during Activity 4 verification testing. Obsolete row versions across active relations confirmed the necessity of running routine vacuuming and statistics updating operations.

2.4 Log Review & Issue Identification

Review of database logs verified that unauthorized access attempts during Activity 4 security testing were recorded by the engine. Logged events included permission denials on update, insertion, and deletion commands attempted under reporting credentials, blocked database creation requests, and denied cross-database connection attempts. Server background processes completed cleanly without system crashes or storage warnings.

2.5 Routine Maintenance Operations

Routine maintenance was conducted by executing system statistics updating and tuple vacuuming operations across target tables. Initial testing confirmed that non-owner application runtime accounts are denied execution of maintenance commands, proving that maintenance tasks require administrative credentials.

Full database statistics updates executed cleanly under administrative credentials. Standard vacuuming on room, property, lease, and payment scanned the relations, removing obsolete row versions and updating frozen transaction counts. Vacuum processing on maintenance_request removed dead tuples and truncated empty pages, restoring storage efficiency.

Full table rewriting vacuum operations were omitted because such commands acquire exclusive locks that block concurrent application read and write operations. Because relations reside within single disk pages, standard non-blocking vacuuming reclaimed dead tuples effectively without disrupting application availability.

2.6 Query Performance Analysis

Performance analysis was conducted on a prospective tenant query searching available rooms by property and sorting results by monthly rental rate. The query
executed in 0.137 milliseconds with complete buffer cache hit efficiency (5 buffer hits, 100% cache hit ratio). The query planner selected nested loop joins with index lookups over sequential scans due to primary key constraints.

Analysis of a financial management billing query joining lease, tenant_profile, room, and payment via v_lease_summary yielded an execution time of 0.281 milliseconds with zero physical disk reads. The query engine executed chained hash joins within dedicated memory structures without disk spills.

2.7 Prioritized Recommendations & Technical Justification

The current index configuration is appropriate for current operational requirements, as active queries execute under 0.29 milliseconds with complete buffer cache hits. Immediate index additions are unnecessary and would introduce write overhead during new lease creation. If multi-year rental records scale significantly in production, adding a composite B-Tree index on payment(lease_id, payment_date) will optimize financial reporting. Deploying a scheduled weekly background maintenance task will maintain page efficiency during low-traffic periods.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 5)

Column Header	Value
Project	Group 11 - BoardTrack: Boarding House & Rental Management System
Database inspected	boardtrack_db (PostgreSQL 18.6)
Account and role used	postgres (DBA Maintenance), boardtrack_app (Landlord Admin), boardtrack_report_user (Auditor)
Health check result	Healthy (0 conflicts, 0 deadlocks, 9.1 MB footprint, 100% buffer cache hits)
Maintenance performed	ANALYZE; VACUUM (VERBOSE, ANALYZE) on room, property, lease, payment via postgres
Query analyzed	Available rooms lookup with rate sort (Nested loop, 0.137 ms) & Multi-table lease billing summary (Hash joins, 0.281 ms)

Recommendation priority	No action required (Current queries execute in < 0.29 ms with 100% buffer cache hits)
Evidence folder link	https://github.com/Nycri/brewpos/tree/main/activity-5/Evi de nce-act5
Feedback sent to owners	https://github.com/Nycri/brewpos/tree/main/activity-5/Evi de nce-act5
PDF progress document	Yes (05/11/2026)
Last updated	https://github.com/Nycri/brewpos/blob/main/PDF_PROG RE SS_DOCUMENTATION.pdf

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.